This guide assumes you have `openvpn`, `strongswan`, `swanctl`, `dnsmasq`, `nftables`, and `curl` installed on your Debian 13 (Trixie) machine.

Step 1: Establish the Declarative Foundation (`systemd-networkd`)

We start by defining your Virtual Routing and Forwarding (VRF) instances, the IPsec XFRM interface, and the core routing policy ladder.

1. Create the VRF and XFRM Interfaces

Create the following `.netdev` and `.network` files in `/etc/systemd/network/`.
(Note: Create matching `10-vrf102` and `10-vrf104` file pairs for the UK and CH tunnels, changing the table numbers to 258 and 260 respectively).

`/etc/systemd/network/10-vrf101.netdev` (AU OpenVPN)

```
Ini, TOML

[NetDev]
Name=vrf101
Kind=vrf

[VRF]
Table=257
```

`/etc/systemd/network/10-vrf101.network`

```
Ini, TOML

[Match]
Name=vrf101

[Network]
# Leaves the VRF active for the OpenVPN hook
```

`/etc/systemd/network/10-xfrm103.netdev` (US StrongSwan)

```
Ini, TOML

[NetDev]
Name=xfrm103
Kind=xfrm

[XFRM]
InterfaceId=103
```

```
[Link]
MTUBytes=1280
```

`/etc/systemd/network/10-xfrm103.network`

```
Ini, TOML

[Match]
Name=xfrm103

[Network]
VRF=vrf103

[RoutingPolicyRule]
Table=259
```

2. Define the Base Policy Routing Ladder

`/etc/systemd/network/10-policy-routing.network`

```
Ini, TOML

[Match]
Name=lo

# Priorities 10/11: Physical Gateway Egress
[RoutingPolicy]
Priority=10
OutgoingInterface=eth0
Table=main

[RoutingPolicy]
Priority=11
OutgoingInterface=wlan0
Table=main

# Priorities 20-50: Strict Domain Rules
[RoutingPolicy]
Priority=20
FirewallMark=0x101
Table=257

[RoutingPolicy]
Priority=30
FirewallMark=0x102
Table=258
```

```
[RoutingPolicy]
Priority=40
FirewallMark=0x103
Table=259

[RoutingPolicy]
Priority=50
FirewallMark=0x104
Table=260
```

3. Setup the Fallback Drop-in Directories

Run these commands to build the fallback options:

```
Bash

sudo mkdir -p /etc/systemd/network/fallbacks/
sudo mkdir -p /etc/systemd/network/10-policy-routing.network.d/

# Create the 5 fallback states
echo -e "[RoutingPolicy]\nPriority=60\nTable=257" | sudo tee /etc/systemd/network/fallb
echo -e "[RoutingPolicy]\nPriority=60\nTable=258" | sudo tee /etc/systemd/network/fallb
echo -e "[RoutingPolicy]\nPriority=60\nTable=259" | sudo tee /etc/systemd/network/fallb
echo -e "[RoutingPolicy]\nPriority=60\nTable=260" | sudo tee /etc/systemd/network/fallb
echo -e "[RoutingPolicy]\nPriority=60\nTable=main" | sudo tee /etc/systemd/network/fall
```

4. Enforce the Default Boot State

`/etc/tmpfiles.d/routing-fallback.conf`

```
Plaintext

L+ /etc/systemd/network/10-policy-routing.network.d/fallback.conf - - - - /etc/systemd/
```

Step 2: Configure Domain Harvesting and Firewall

1. Configure `dnsmasq`

`/etc/dnsmasq.d/stealth-routing.conf`

```
Plaintext

nftset=/netflix.com.au/4#ip#domain_routing#au_targets
nftset=/bbc.co.uk/4#ip#domain_routing#uk_targets
```

```
nftset=/hulu.com/4#ip#domain_routing#us_targets
nftset=/admin.ch/4#ip#domain_routing#ch_targets
```

2. Configure `nftables` Base Framework
`/etc/nftables.conf` (Append this table to your existing config)

```
Plaintext

table ip domain_routing {
    set au_targets { type ipv4_addr; flags dynamic, timeout; timeout 1h; }
    set uk_targets { type ipv4_addr; flags dynamic, timeout; timeout 1h; }
    set us_targets { type ipv4_addr; flags dynamic, timeout; timeout 1h; }
    set ch_targets { type ipv4_addr; flags dynamic, timeout; timeout 1h; }

    chain prerouting {
        type filter hook prerouting priority mangle; policy accept;
        ip daddr @au_targets meta mark set 0x101
        ip daddr @uk_targets meta mark set 0x102
        ip daddr @us_targets meta mark set 0x103
        ip daddr @ch_targets meta mark set 0x104
    }
}
```

Step 3: Configure the VPN Daemons

1. The OpenVPN VRF Hook
Create `/etc/openvpn/scripts/vrf-up.sh` and run `sudo chmod +x` on it:

```
Bash

#!/bin/bash
TUN_INDEX="${1#tun}"
VRF_NAME="vrf$TUN_INDEX"
TABLE_ID=$((16#$TUN_INDEX))

ip link set dev "$1" master "$VRF_NAME"
ip link set dev "$1" up
ip route flush table "$TABLE_ID"
ip route add default dev "$1" table "$TABLE_ID"
```

2. OpenVPN Client Overrides (AU, UK, CH)
Add these blocks to your respective `.ovpn` files in `/etc/openvpn/client/` (`au.conf`, `uk.conf`, `ch.conf`). They must have explicit `tun-mtu` and `mssfix` lines so our bash scripts can locate and edit them dynamically.

Inside `au.conf` (Use `tun102` for UK, `tun104` for CH):

```
Plaintext

dev tun101
dev-type tun
proto tcp-client
route-noexec
nobind
script-security 2
up /etc/openvpn/scripts/vrf-up.sh
tun-mtu 1280
mssfix 1024
```

(For `ch.conf`, also ensure `scramble obfuscate` is present).

3. StrongSwan XFRM Config (US)

`/etc/swanctl/swanctl.conf`

```
Plaintext

connections {
    nordvpn-us {
        local_addrs  = %defaultroute
        remote_addrs = US_SERVER_IP
        version = 2
        send_cert = never
        dpd_delay = 300s
        local { auth = eap-mschapv2; eap_id = "Username" }
        remote { auth = pubkey; id = %US_SERVER_HOSTNAME }
        children {
            nordvpn-us {
                local_ts  = 0.0.0.0/0
                remote_ts = 0.0.0.0/0
                if_id_in  = 103
                if_id_out = 103
                updown = /usr/lib/ipsec/_updown iptables
            }
        }
    }
}
secrets {
    eap-nordvpn { id = "Username"; secret = "YourPassword" }
}
```

Step 4: The Custom Bash Utilities

Create these two files in `/usr/local/bin/` and make them executable (`sudo chmod +x /usr/local/bin/vpn` and `sudo chmod +x /usr/local/bin/nested`).

1. The `vpn` Command (Fast Fallback Switcher)

`/usr/local/bin/vpn`

Bash

```
#!/bin/bash
if [ "$EUID" -ne 0 ]; then echo "Please run: sudo vpn ${@}"; exit 1; fi

apply_vpn() {
    ln -sf "/etc/systemd/network/fallbacks/${1}" "/etc/systemd/network/10-policy-routi
    networkctl reload
    echo "✅ Fallback switched to: ${2}"
}

show_usage() {
    echo "Usage: vpn [AU|UK|US|CH|XX]"; exit 1
}

if [ "$#" -eq 0 ]; then
    echo "1) AU  2) UK  3) US  4) CH  5) XX (Direct WAN)"
    read -p "Select fallback [1-5]: " choice
    case "$choice" in
        1) apply_vpn "au.conf" "AU VPN" ;;
        2) apply_vpn "uk.conf" "UK VPN" ;;
        3) apply_vpn "us.conf" "US VPN" ;;
        4) apply_vpn "ch.conf" "CH VPN" ;;
        5) apply_vpn "main.conf" "Direct WAN" ;;
        *) echo "Error: Invalid selection."; exit 1 ;;
    esac
elif [ "$#" -eq 1 ]; then
    case "${1^^}" in
        AU) apply_vpn "au.conf" "AU VPN" ;;
        UK) apply_vpn "uk.conf" "UK VPN" ;;
        US) apply_vpn "us.conf" "US VPN" ;;
        CH) apply_vpn "ch.conf" "CH VPN" ;;
        XX) apply_vpn "main.conf" "Direct WAN" ;;
        *) echo "Error: Unrecognised argument '${1}'."; show_usage ;;
    esac
else
    echo "Error: Too many arguments."; show_usage
fi
```

2. The `nested` Command (Idempotent Topology Orchestrator)
`/usr/local/bin/nested`

Bash

```
#!/bin/bash
if [ "$EUID" -ne 0 ]; then echo "Please run: sudo nested ${@}"; exit 1; fi
if [ "$#" -ne 1 ]; then echo "Usage: nested [US|CH|OFF]"; exit 1; fi

arg="${1^^}"

echo "Tearing down all existing VPN connections..."
systemctl stop openvpn-client@au openvpn-client@uk openvpn-client@ch strongswan-starter
ip rule del priority 5 2>/dev/null || true
nft 'flush chain ip domain_routing postrouting' 2>/dev/null || true

AU_IP=$(grep -m 1 "^remote " /etc/openvpn/client/au.conf | awk '{print $2}')
UK_IP=$(grep -m 1 "^remote " /etc/openvpn/client/uk.conf | awk '{print $2}')
CH_IP=$(grep -m 1 "^remote " /etc/openvpn/client/ch.conf | awk '{print $2}')
US_IP=$(grep -m 1 "remote_addrs" /etc/swanctl/swanctl.conf | grep -oE '[0-9]+\.[0-9]+\

set_ovpn_mtu() {
    sed -i -E "s/^tun-mtu .*/tun-mtu $2/" "/etc/openvpn/client/${1}.conf"
    sed -i -E "s/^mssfix .*/mssfix $3/" "/etc/openvpn/client/${1}.conf"
}
set_xfrm_mtu() { sed -i -E "s/^MTUBytes=.*/MTUBytes=$1/" /etc/systemd/network/10-xfrm1

apply_xfrm_mangle() {
    nft 'add chain ip domain_routing postrouting { type filter hook postrouting priorit
    nft 'add rule ip domain_routing postrouting oifname "xfrm103" tcp flags syn tcp opt
}

wait_for_outer_tunnel() {
    local max_attempts=10
    local delay=5
    echo "----------------------------------------"
    echo "Actively polling nordvpn.com for protection status..."
    for ((i=1; i<=max_attempts; i++)); do
        echo "Attempt $i/$max_attempts: Sleeping ${delay}s to allow tunnel to stabilise
        sleep $delay
        STATUS_RAW=$(curl -sL --max-time 5 https://nordvpn.com | tr -d '\n' | grep -Po
        STATUS_CLEAN=$(echo "$STATUS_RAW" | sed 's/<[^>]*>//g' | sed 's/^[ \t]*//;s/[ '

        if [ -n "$STATUS_CLEAN" ]; then
            echo "Status Line: $STATUS_CLEAN"
            if echo "$STATUS_CLEAN" | grep -iq "Unprotected"; then
                echo "Result: Connection is UNPROTECTED."
```

```
            elif echo "$STATUS_CLEAN" | grep -iq "Protected"; then
                echo "✅ Outer tunnel VERIFIED as Protected!"
                echo "----------------------------------------"
                return 0
            fi
        else
            echo "Result: Could not reach or parse nordvpn.com."
        fi
    done
    echo "⚠️ CRITICAL TIMEOUT: Outer tunnel failed to verify."
    exit 1
}

case "$arg" in
    US)
        echo "Configuring Topology: US Outer (AU, UK, CH nested inside)..."
        ip rule add to $AU_IP priority 5 lookup 259
        ip rule add to $UK_IP priority 5 lookup 259
        ip rule add to $CH_IP priority 5 lookup 259
        set_ovpn_mtu "au" 1280 1024; set_ovpn_mtu "uk" 1280 1024; set_ovpn_mtu "ch" 128
        set_xfrm_mtu 1400
        ln -sf "/etc/systemd/network/fallbacks/us.conf" "/etc/systemd/network/10-policy
        apply_xfrm_mangle
        ;;
    CH)
        echo "Configuring Topology: CH Outer (AU, UK, US nested inside)..."
        ip rule add to $AU_IP priority 5 lookup 260
        ip rule add to $UK_IP priority 5 lookup 260
        ip rule add to $US_IP priority 5 lookup 260
        set_ovpn_mtu "au" 1280 1024; set_ovpn_mtu "uk" 1280 1024; set_ovpn_mtu "ch" 140
        set_xfrm_mtu 1280
        ln -sf "/etc/systemd/network/fallbacks/ch.conf" "/etc/systemd/network/10-policy
        apply_xfrm_mangle
        ;;
    OFF)
        echo "Configuring Topology: 5 Independent Parallel Connections..."
        set_ovpn_mtu "au" 1280 1024; set_ovpn_mtu "uk" 1280 1024; set_ovpn_mtu "ch" 128
        set_xfrm_mtu 1280
        ln -sf "/etc/systemd/network/fallbacks/main.conf" "/etc/systemd/network/10-poli
        apply_xfrm_mangle
        ;;
    *)
        echo "Error: Unrecognised argument '${1}'."; exit 1 ;;
esac

echo "Synchronising systemd-networkd..."
networkctl reload
```

```
if [ "$arg" == "US" ]; then
    echo "Starting OUTER tunnel (US IPsec)..."
    systemctl start strongswan-starter
    swanctl --load-all
    wait_for_outer_tunnel
    echo "Starting INNER nested tunnels (AU, UK, CH)..."
    systemctl start openvpn-client@au openvpn-client@uk openvpn-client@ch
elif [ "$arg" == "CH" ]; then
    echo "Starting OUTER tunnel (CH Obfuscated OpenVPN)..."
    systemctl start openvpn-client@ch
    wait_for_outer_tunnel
    echo "Starting INNER nested tunnels (AU, UK, US)..."
    systemctl start openvpn-client@au openvpn-client@uk strongswan-starter
    swanctl --load-all
elif [ "$arg" == "OFF" ]; then
    echo "Starting all independent connections in parallel..."
    systemctl start openvpn-client@au openvpn-client@uk openvpn-client@ch strongswan-st
    swanctl --load-all
fi
echo "========================================"
echo "✅ Network topology successfully rebuilt!"
```

Step 5: Start the Architecture

Disable any old conflicting networking tools managing your `lo` interface (if applicable) and hand the keys to `systemd-networkd`:

```
Bash

sudo systemctl enable systemd-networkd
sudo systemctl start systemd-networkd
```

With `systemd-networkd` running, your VRFs and base rules are active. You can now pull the trigger and orchestrate the entire setup cleanly by running:

```
Bash

sudo nested off
```